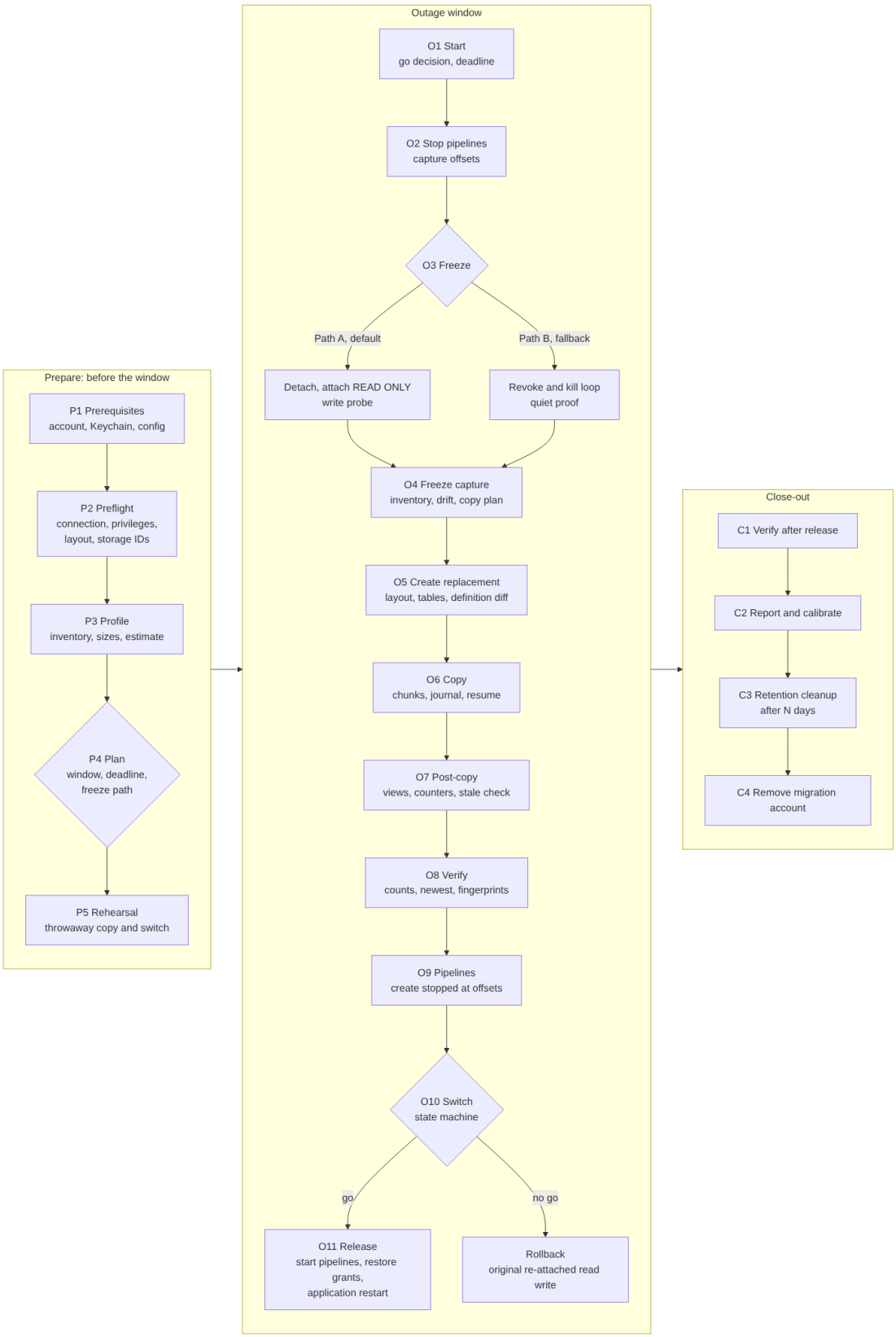
SingleStore Database Rebuild Kit

Step guide: the README and the step catalog

Author: John Bergamini

Version 1.0.0

Flow of the steps



SingleStore Database Rebuild Kit

Author: John Bergamini. Version 1.0.0. Proprietary; see LICENSE.

The kit rebuilds a SingleStore Helios database into a new partition layout and cuts it over under its original name inside one outage window. The first outage step makes the source read only. Everything after that is a copy, a verification and a switch, each behind a gate, each restartable from a terminal.

Who runs what

Identity	Who	Steps
runner	A SQL account the kit creates at P1 and drops at C4	P2, P3 tables, O4 to O8, C1, C2, watch
owner	The workspace admin SQL user, or the Portal identity in the owner notebook	P1 account, P3 pipelines, O2, O3, O9, O10, O11, C3, C4, lockout
operator	The person running the kit	every command; records the gates a person decides
customer	The application owner	stops the application (if asked), hands over pipeline credentials, restarts the application at O11

The runner copies. The owner freezes, switches and releases. With an owner profile in the configuration the kit runs the admin steps itself; without one it prints each admin block and waits for `s2rebuild gate`.

Install

Python 3.10 or later on macOS or Linux. The Keychain password source is macOS only; elsewhere use `password_env`.

```
python3 -m venv .venv
.venv/bin/pip install -e .
.venv/bin/s2rebuild --help
```

Run the offline tests with `.venv/bin/pip install pytest` and `.venv/bin/python -m pytest`. No test opens a network connection.

Configure

1. Copy the example: `cp examples/config.example.yaml customer/config.yaml`.
2. Set the workspace host, the database names and the target layout. Every key is in docs/CONFIG.md.
3. Store each password in the macOS Keychain (`security add-generic-password -a "$USER" -s s2rebuild-runner -w`) or name an environment variable with `password_env`. The kit never prints or writes a password.
4. Everything a run writes stays under `customer/`. The rest of the folder is the static tree.

Steps

Each step prints what it did and the gate it needs. Exit codes: 0 done, 1 refused, 2 incomplete (rerun to resume), 3 verification mismatch, 4 gate pending or failed. The full catalog with inputs, outputs, statements and recovery is docs/STEPS.md.

Prepare (before the window)

1. P1 Prerequisites. Print the runner account block with `s2rebuild account-sql` and have the admin run it, or run it with `s2rebuild account-sql --execute`. Then run `s2rebuild check`. Gate: `check` passes; record it with `s2rebuild gate P1 --pass --note TEXT`.
2. P2 Preflight. `s2rebuild check` connects as the runner (and the owner), checks every runner privilege by name, records the source STORAGE_ID, confirms the replacement and retained names are free and the compute size fits the partition count. Gate: every line ok.
3. P3 Profile. `s2rebuild profile` captures the inventory (tables, keys, sizes, accounts, sessions, views, pipelines with the owner profile) and writes `profile.xlsx` with the estimate and the window fit. Gate: the estimate fits the window.
4. P4 Plan. `s2rebuild plan` prints the plan sheet: window, abort deadline, freeze path, step owners, pipeline credential route. Gate: the decisions are recorded with `s2rebuild gate P4 --pass --note TEXT`.
5. P5 Rehearsal. `s2rebuild rehearse` builds throwaway `s2rebuild_rehearsal_` databases, copies them with a SIGKILL test and a resume, verifies, freezes, switches and drops what it created. The killed chunk leaves a leaf lock for 15 to 20 minutes, so plan 30 minutes. Gate: P5 passed (the switch needs the owner profile).

Outage window

6. O1 Start. `s2rebuild run --from O1` records the window and the abort deadline and waits for the go decision: `s2rebuild gate O1 --pass --note TEXT`. Then `s2rebuild run` continues.
7. O2 Stop pipelines. The owner stops every pipeline and the kit captures every definition and offset again after the stop. These are the offsets O9 replays. Gate: no pipeline Running; capture written.
8. O3 Freeze. Path A (default): DETACH, ATTACH READ ONLY, a write probe that must be refused. A DETACH that waits longer than `freeze.detach_wait_seconds` is cancelled and issued again with KILL. Path B: `s2rebuild lockout` revokes write privileges and kills sessions until the quiet proof passes. After a notebook or hand run, `s2rebuild step O3 --redo` runs the probe and records freeze_time. Gate: probe refused and READONLY (A), or quiet proof passed (B).
9. O4 Freeze capture. `s2rebuild profile --step O4` captures the inventory after the freeze and reports drift against P3. Gate: drift reviewed.
10. O5 Create replacement. `s2rebuild run --from O4` continues: the replacement is created with the target layout and every table definition is compared with the capture; its STORAGE_ID is recorded.
11. O6 Copy. Server-side INSERT ... SELECT in hash buckets, journaled, resumable. Stop it only with `s2rebuild pause` or the abort deadline: a killed client leaves a prepared transaction holding a leaf table lock for 15 to 20 minutes. Gate: every chunk done; per-table count proof all match.
12. O7 Post-copy. Views, AUTO_INCREMENT counters and the stale check (counts and newest values) with re-copy of anything that moved. Gate: zero mismatches.
13. O8 Verify. Counts, newest values and grouped fingerprints within the time budget. Gate: tiers 1 and 2 pass for every table; tier 3 within budget or accepted with `--yes`.
14. O9 Pipelines. Export the credential variables for this step only, run it, then unset them. The kit recreates every pipeline in the replacement, stopped, at the O2 offsets. Route customer: the customer runs the placeholder file and the kit checks. Gate: count, none Running, offsets equal the O2 capture.
15. O10 Switch. A state machine: retained copy attached read only first, then the replacement as a branch under the live name. Type the live name when asked (or pass `--yes`). Gate: the live name resolves to the

new branch with the target layout and the capture counts.

16. O11 Release. Retention check per partition, grants restored, pipelines started; the customer restarts the application. Gate: application writing.

Close-out

17. C1 Verify after release. `s2rebuild step C1`: writer watch, count changes, pipelines Running with offsets advancing, no errors, grants compared. Gate: all ok.
18. C2 Report and calibrate. `s2rebuild step C2` writes `final.xlsx` and `run-report.md` and refits the throughput model into `customer/baseline.json`.
19. C3 Retention cleanup. After `closeout.retain_days`: `s2rebuild step C3` drops the retained branch and the replacement. Delete the detached original in the Portal by STORAGE_ID.
20. C4 Remove the migration account. `s2rebuild step C4` drops the runner account; delete its Keychain item.

Troubleshooting

- Lock waits (error 1205): the kit serializes the table, probes the lock and retries after `copy.lock_retry_seconds`. A lock held by a killed client clears in 15 to 20 minutes. If it does not clear, `s2rebuild step O6 --fresh-table TABLE` loads the table into a new table and renames it.
- Detach waits: a DETACH blocked by writers is cancelled after `freeze.detach_wait_seconds` and issued with KILL when `freeze.detach_kill` is true. With `detach_kill: false` the step stops; run `s2rebuild lockout` as a bridge, then `s2rebuild step O3 --redo`.
- Grants restored by another session: the lockout logs `GRANT RESTORED BY ANOTHER SESSION` and revokes again on the next pass. Find the job that restores them before the night.
- Name collisions: names are not unique on Helios. Every ATTACH names a STORAGE_ID. `s2rebuild check` refuses a replacement or retained name that already exists.
- Pipelines the application recreates: list them in `databases[].app_managed_pipelines`. They are stopped at O2, never replayed, and C1 reports whether they came back.
- An admin step stopped at a check: the error names the check, the expected value and the observed value. Fix the cause and rerun the step; the switch re-enters from the live attachment state.

Documents

- docs/STEPS.md: every step in full.
- docs/CONFIG.md: every key, flag, variable and exit code.
- docs/CODE-GUIDE.md: modules, data flow, decisions you can change.
- docs/LESSONS.md: Helios facts and run lessons.
- docs/ARCHITECTURE.md: the design.
- `docs/s2rebuild-guide.pdf`: this README and STEPS with the flow diagram, built by `docs/build_pdf.py`.

Steps

Author: John Bergamini

One section per step. Each lists the purpose, the identity, inputs, outputs, the statements the kit issues, the gate, how to re-enter and what to do when it fails. Admin steps print their block (also written to `customer/runs/<run>/<db>/owner-sql/<step>.sql`, mode 0600); with the owner profile the kit executes it statement by statement and stops at the first check that does not return the expected value. Every statement passes the step's guard first.

Commands below assume `s2rebuild` on the path and `customer/config.yaml` as the configuration. Add `--db NAME` when several databases are configured.

P1 Prerequisites

- Purpose: an account for the copy, its password stored, the configuration written.
- Identity: owner (account) and the operator.
- Inputs: `examples/config.example.yaml`, the admin.
- Outputs: the runner account with the privileges of section 5 of the architecture; a Keychain item or variable per identity; `customer/config.yaml`.
- Statements: `s2rebuild account-sql` prints CREATE USER and GRANT with checks before and after; `--execute` runs them with the owner profile and injects the password at execution only.
- Gate: `s2rebuild check` passes. Record: `s2rebuild gate P1 --pass --note TEXT`.
- Re-enter: rerun `account-sql`; its first check stops when the account already exists.
- When it fails: a missing privilege is named by `check`; grant it and rerun.

P2 Preflight

- Purpose: prove the connection, privileges, layout, storage id, names and compute size before anything else.
- Identity: runner, plus the owner when configured.
- Inputs: the configuration.
- Outputs: the source STORAGE_ID in the journal, the engine version, a preflight record.
- Statements: SELECT only: CURRENT_USER and version, USER_PRIVILEGES for the runner, DISTRIBUTED_DATABASES, MV_CLOUD_ATTACHED_DATABASES, MV_NODES leaf cores, PIPELINES count (owner).
- Gate: every line ok: all runner privileges present, source reported with a STORAGE_ID, replacement and retained names free, leaf cores times 8 at or above the target partitions (when MV_NODES is readable; otherwise confirm in the Portal).
- Re-enter: `s2rebuild check` any time; O1 repeats it.
- When it fails: each failed line says what to change (grant, rename, compute size).

P3 Profile

- Purpose: know the database: tables, keys, sizes, accounts, sessions, views, routines, pipelines; estimate the copy.

- Identity: runner; owner for pipelines.
- Inputs: the configuration, `customer/baseline.json` when present.
- Outputs: `inventory/P3/inventory.json`, `tables.csv`, `definitions/`, `pipelines/` (0600, CREDENTIALS replaced by <<CREDENTIALS>>), `reports/profile.xlsx` with the estimate, bands and window fit.
- Statements: SELECT and SHOW only (SHOW CREATE PIPELINE ... EXTENDED with the owner profile).
- Gate: the estimate at the high band fits before the abort deadline.
- Re-enter: `s2rebuild profile` again; `--exact-counts` for COUNT(*) per table.
- When it fails: `DOES NOT FIT` exits 4; widen the window, raise workers, or split the databases across nights (`--all-databases` ranks them).

P4 Plan

- Purpose: the decisions people make before the night.
- Identity: the people named in the plan.
- Inputs: `s2rebuild plan`.
- Outputs: a recorded gate.
- Statements: none.
- Gate: freeze path, window, abort deadline, the named owner for "application stopped", the pipeline credential route and handover, the retained name and retention days, recorded in the configuration.
- Re-enter: edit the configuration and print the plan again.

P5 Rehearsal

- Purpose: run the exact steps on throwaway databases first, including a hard kill and the switch.
- Identity: runner and owner.
- Inputs: `--rows` (default 1000000), `--prefix` (must start with `s2rebuild_rehearsal_`).
- Outputs: `customer/runs/<run>/rehearsal/timings.json` with every step time, the interrupted and reconciled chunks and the Portal cleanup note.
- Statements: CREATE and DROP DATABASE on the prefixed names only, the copy, verify and switch statements on them.
- Gate: P5 passed: copy, SIGKILL and resume (exactly the killed chunk reconciled), verify, freeze and switch completed. Without the owner profile the switch is not rehearsed and P5 stays pending.
- Re-enter: `s2rebuild rehearse --cleanup --prefix PREFIX --yes`, then rehearse again.
- When it fails: the kit drops only the databases it created in that run.

O1 Start

- Purpose: start the window with a recorded go decision.
- Identity: runner and the operator.
- Inputs: the window in the configuration.
- Outputs: names, window, abort deadline (end minus restart allowance, buffer and the longest modeled chunk) in the journal.
- Statements: none beyond reading the capture.

- Gate: deadline in the future, P2 clean, rehearsal evidence present, go recorded: `s2rebuild gate O1 --pass --note TEXT`.

O2 Stop pipelines

- Purpose: stop every pipeline and record the final bookmark of each.
- Identity: owner.
- What a pipeline and an offset are: a pipeline is a server-side ingestion job reading a source (a Kafka topic, an S3 prefix and others) into one table. Its offset is its bookmark per source partition. A pipeline recreated with its recorded offsets resumes where the old one stopped: no message skipped, none loaded twice.
- Outputs: `inventory/O2/` with every definition (credentials replaced by the placeholder) and the offsets read after the stop; journal pipeline rows.
- Statements: STOP PIPELINE per pipeline (skipped for one already stopped), a check that none is Running, then the capture. The stop list is the earlier capture plus any pipeline the live view shows.
- Gate: no pipeline Running; every captured pipeline Stopped. A pipeline that runs again before the capture refuses the gate.
- Notebook route: the O2 cell's result carries the capture; `s2rebuild gate O2 --pass --note TEXT --evidence FILE` imports it.

O3 Freeze

- Purpose: make the source read only for every account.
- Identity: owner; the write probe runs as the runner.
- Path A (default): checks (READWRITE, recorded STORAGE_ID, no pipeline Running), DETACH, check detached, ATTACH ... STORAGEID ... READ ONLY, checks (READONLY, same STORAGE_ID), then the write probe: a DELETE for a value that cannot exist on a string column (or an integer key below its minimum). Pass means the server refused it as read only. Then freeze_time and, with the owner, a grants reference for C1.
- DETACH waits: after `freeze.detach_wait_seconds` the kit cancels the waiting DETACH on its own connection and issues DETACH ... KILL when `freeze.detach_kill` is true; otherwise it stops and the lockout serves as a bridge.
- Path B: `s2rebuild lockout`: snapshot every write-capable account at every scope (global, database, table) to the journal and `grants-snapshot.json` (0600), revoke exactly those privileges, hold connections, and on every pass re-revoke grants restored by another session and kill every session of those accounts (idle ones too) from the connection that sees it. After the configured quiet passes the quiet proof runs; it records freeze_time when pipelines are stopped, no foreign session is active and counts and newest values hold for a minute.
- Gate: Path A, probe refused and READONLY; Path B, quiet proof passed.
- Re-enter: `s2rebuild step O3 --redo`. When the source is already READONLY on the recorded STORAGE_ID the kit only checks, probes and records freeze_time, with or without the owner profile.

O4 Freeze capture

- Purpose: the inventory the copy uses, taken after the freeze.
- Identity: runner.
- Inputs: `s2rebuild profile --step O4`.

- Outputs: `inventory/O4/`, `drift.json` against P3, the copy plan.
- Gate: drift reviewed.

O5 Create replacement

- Purpose: the replacement database with the target layout and identical table definitions.
- Identity: runner.
- Statements: CREATE DATABASE ... PARTITIONS ... SUB_PARTITIONS, CREATE TABLE ... LIKE per table, SHOW CREATE TABLE.
- Outputs: the replacement STORAGE_ID in the journal (the switch attaches it by that id).
- Gate: layout and durability verified; every definition equals the capture (only the AUTO_INCREMENT counter may differ).

O6 Copy

- Purpose: copy every row.
- Identity: runner.
- Statements: INSERT INTO replacement SELECT FROM source WHERE CRC32(key) MOD B = k per bucket; reconciliation DELETE of a started or failed bucket before it is re-inserted.
- Behaviour: a queue per table; a table that hits a lock wait (1205) is serialized and probed; the journal is written before each statement; the abort deadline and `s2rebuild pause` stop new chunks while running ones finish. Never hard-stop the copy: a killed client leaves a prepared transaction holding a leaf table lock for 15 to 20 minutes.
- Gate: every chunk done; per-table count proof all match.
- Re-enter: `s2rebuild run --from O6` resumes; `--retry-failed` reruns parked chunks; `s2rebuild mark-recopy TABLE [--bucket N]` redoes buckets; `s2rebuild step O6 --fresh-table TABLE` loads a locked table into a new table and renames it.

O7 Post-copy

- Purpose: views, counters and the stale check.
- Identity: runner.
- Statements: CREATE VIEW (DEFINER removed, original definer reported), ALTER TABLE ... AUTO_INCREMENT = MAX + 1 and AGGREGATOR SYNC AUTO_INCREMENT, counts and newest values on both sides.
- Gate: views created, counters set, zero stale mismatches after re-copy.

O8 Verify

- Purpose: prove the copy.
- Identity: runner.
- Tiers: 1 counts both sides; 2 newest time value and key range; 3 grouped fingerprints (key hash and value hash per group, mapped to buckets), smallest tables first within `verify.fingerprint_budget_minutes`, sampled beyond it.
- Outputs: verification rows, `verified_at_utc` per table, the final workbook and run report.

- Gate: tiers 1 and 2 pass everywhere; tier 3 within budget, or a sampled result accepted with `s2rebuild step O8 --yes`.
- When it fails: exit 3; the mismatching buckets are named for `mark-recopy`.

O9 Pipelines

- Purpose: every pipeline exists in the replacement, stopped, at the offsets recorded at O2.
- Identity: owner.
- Credentials: the server hides CREDENTIALS from every identity, so the customer supplies them. Route kit: the operator exports the variable per source type (or per pipeline) named in `pipelines.credentials_env` and `pipelines.per_pipeline_env` for this step only and unsets it afterwards; the kit injects the value as a SQL literal at execution and masks it in every log line. In the notebook the cell asks for each value with a hidden prompt. Route customer: the customer runs `pipelines/recreate-with-placeholders.sql` from the O2 capture inside the replacement, filling in CREDENTIALS; the kit runs only the checks.
- Statements: USE replacement, CREATE PIPELINE per pipeline (skipped when it exists), its ALTER PIPELINE ... SET OFFSETS statements. Never START.
- Gate: count equals the capture minus application-managed pipelines; none Running; offsets in the replacement equal the O2 capture partition by partition.
- Refusals: only the O2 capture is replayed (an earlier capture's offsets were still moving); a definition that names the source database is refused.

O10 Switch

- Purpose: the live name resolves to the replacement, the original stays readable.
- Identity: owner.
- S0 (decision): O8 and O9 passed, every table verified after freeze_time, both storage ids recorded, time left before the abort deadline; the operator types the live name (or passes `--yes`).
- S1: checks (source on its recorded STORAGE_ID, attach type for the freeze path, no foreign active session, replacement READWRITE on its recorded id, retained name free), DETACH (with the wait and KILL rule), checks.
- S2: ATTACH source AS retained STORAGEID ... READ ONLY, checks, one small table count.
- S3: checks, ATTACH replacement AS live STORAGEID ..., checks: READWRITE on a new STORAGE_ID, target layout, table, view and pipeline counts, three smallest tables against the journal.
- Re-enter: the position comes from the attachment views. A rerun repeats the after-checks of the step it lands behind, so a failed check never turns into a passed switch.
- Rollback (before O11 only): R1 detach the live branch, R2 re-attach the original read write by STORAGEID, R3 drop the retained copy (optional). It resumes from any position behind S1 and is refused once O11 released writers or C1 saw application writes.

O11 Release

- Purpose: writers back on the live name.
- Identity: owner and the customer.
- Retention check: for every captured partition the recorded offset must not be below the current EARLIEST_OFFSET, and the partition must exist. A gap means messages were lost to the source's

retention; decide EARLIEST or LATEST per pipeline and record it in the gate note.

- Statements: GRANT restores (each checked) from any unrestored lockout snapshot, START PIPELINE per expected pipeline (skipped when Running), SET GLOBAL query_parallelism_per_leaf_core when configured.
- Gate: every expected pipeline Running; grants equal the snapshot; the application restarted and writing. Recording the gate by hand stamps the release time.

C1 Verify after release

- Identity: runner; owner for pipelines and grants.
- Checks: writer watch, count changes on the largest tables (two samples a minute apart and against the copy), pipelines Running with offsets advancing where data waits, no pipeline errors since the release, grants equal to the snapshot (Path B) or the reference (Path A). Application writes are recorded; from then on a rollback is a new freeze and a reconciliation.
- Gate: all ok. Without the owner profile the pipeline and grants checks stay pending.

C2 Report and calibrate

- Identity: runner.
- Outputs: `reports/final.xlsx`, `reports/run-report.md`, `customer/baseline.json` refitted from the run's first-pass chunks.
- Gate: reports written and the baseline recalibrated; with too few chunks to fit, the shipped baseline stays and the gate waits for a person.

C3 Retention cleanup

- Identity: owner.
- When: after `closeout.retain_days` from the release; the step refuses earlier.
- Statements: checks, DROP DATABASE retained, DROP DATABASE replacement, checks that the live name still resolves to the live branch.
- Then: delete the detached original storage in the Portal by STORAGE_ID; keep the evidence folder.

C4 Remove the migration account

- Identity: owner.
- Statements: a check that the account still holds privileges, DROP USER, a check that zero privileges remain.
- Then: delete the Keychain item that held the runner password.